HPML Final Project · Spring 2026

Towards Multi-Turn Dialog Systems for Industrial Asset Operations

Improved multi-turn diagnostics with memory-aware replanning and parallel tool execution, increasing task success rate, reducing tool calls, and improving robustness.

Team 5
Rujing Li · rl3641
Yitong Bai · yb2636
Chengrui Li · cl4750
Rui Li · rl3586

GitHub: https://github.com/Coderlicr/Multi-Turn-AssetOps
Tracking: https://wandb.ai/rl3641-col/multi-turn-assetops?nw=lk02esfc7g

Dr. Kaoutar El Maghraoui · Columbia University · Week of May 4, 2026

Motivation & Problem

Why this matters

Real-world Setting

Industrial O&M QA is **inherently multi-turn and iterative**. Users continuously **refine queries, ask follow-ups**, and expect dynamic tool invocation.

Cost / Latency Pressure

Tool invocation is the **primary bottleneck**. Reusing **intermediate results** is critical for reducing latency, cost, and system instability.

Naive Baseline Limitations

Our baseline is Plan-Execute single-agent architecture, which:

- **only** support single-turn interaction;
- struggle to **reliably plan** tool usage and suffer from parameter **hallucination**;
- experience context explosion under failures.

Problem Statement

To meet the demands of real-world Industrial O&M QA, we identify key limitations in baseline architecture when applied to **multi-turn, tool-centric workloads**, including insufficient task planning, fragile tool invocation, and ineffective reuse of intermediate artifacts.

Interaction: Single-turn ⟶ **Multi-turn**

Architecture: Single Agent ⟶ **Multi-Agent (Supervisor-Specialist)**

Goal

In complex multi-turn dialogue scenarios:

- **improve** task success rate and tool reliability;
- **mitigate** tool invocation bottlenecks through cross-turn result reuse.

Technical Approach — System, Model & Data

Model

LLM Backend: LiteLLM / IBM WatsonX for reasoning and planning.

Forecasting: TinyTimeMixer / Granite-TSFM for time-series forecasting.

Anomaly Detection: Conformal anomaly detection for sensor scoring.

Dataset

16 multi-turn industrial diagnosis scenarios developed through discussions with SME

IoT sensor time-series data for MAIN site assets: Chiller 3, Chiller 4, Chiller 6, and Chiller 9

Operational records including work orders, events, alerts, and failure-code mappings

Runtime / Hardware

Database: Dockerized CouchDB

Servers: Python MCP servers

Framework: TSFM (PyTorch supported)

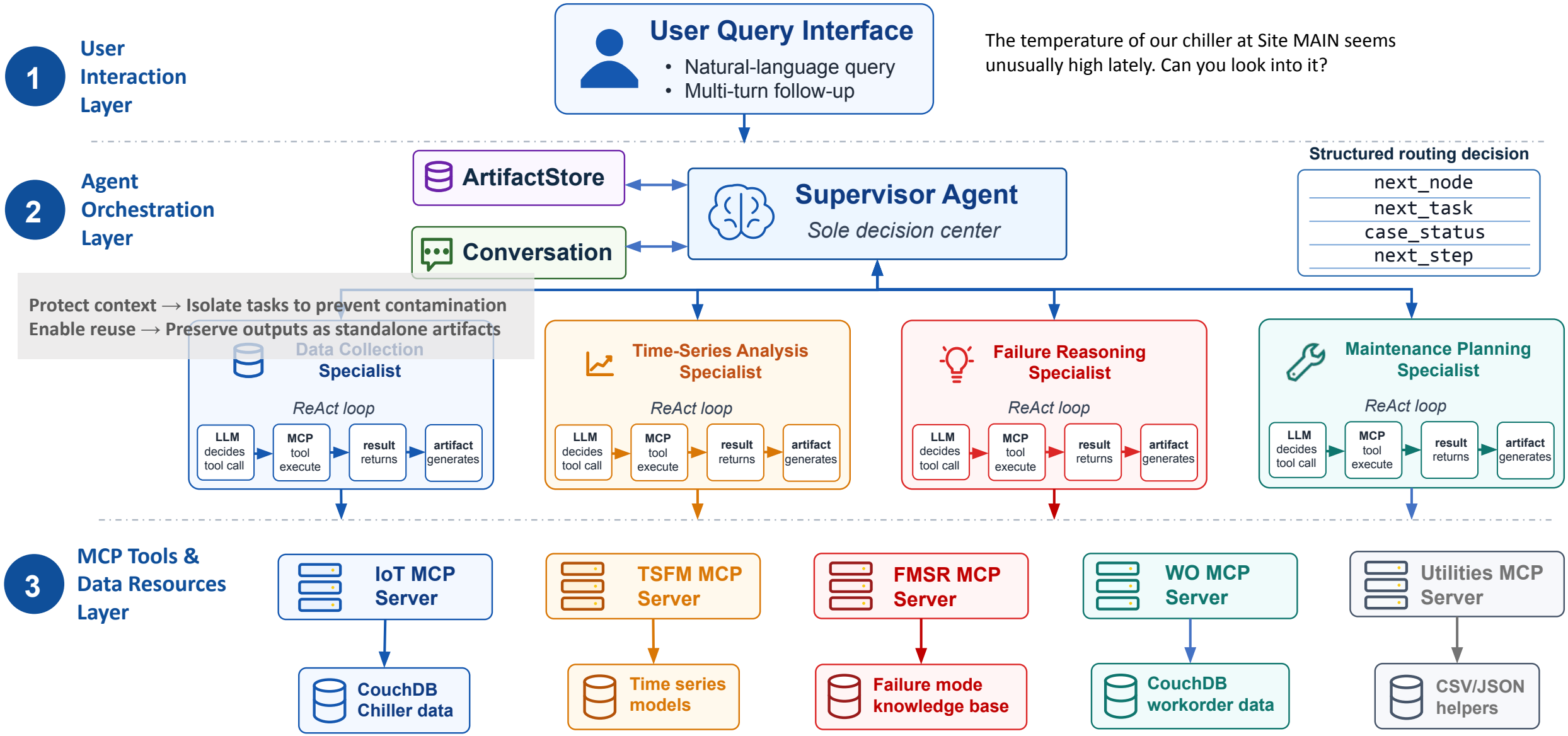
LLM Access: Remote calls via WatsonX API

LLM inference is handled remotely when using WatsonX backend.

System Architecture

Supervisor-Specialist multi-agent workflow: supervisor routes user requests to specialist agents for data collection, time-series analysis, failure reasoning, and maintenance planning. Tools are exposed through MCP servers.

Supervisor-Specialist Multi-Agent System for Industrial Asset Operations



Technical Approach — Optimizations Applied

1. Artifact Reuse for Multi-Turn Memory

Reuses structured specialist artifacts across turns to maintain context and avoid repeated tool calling or analysis.

2. Parallel Tool Execution

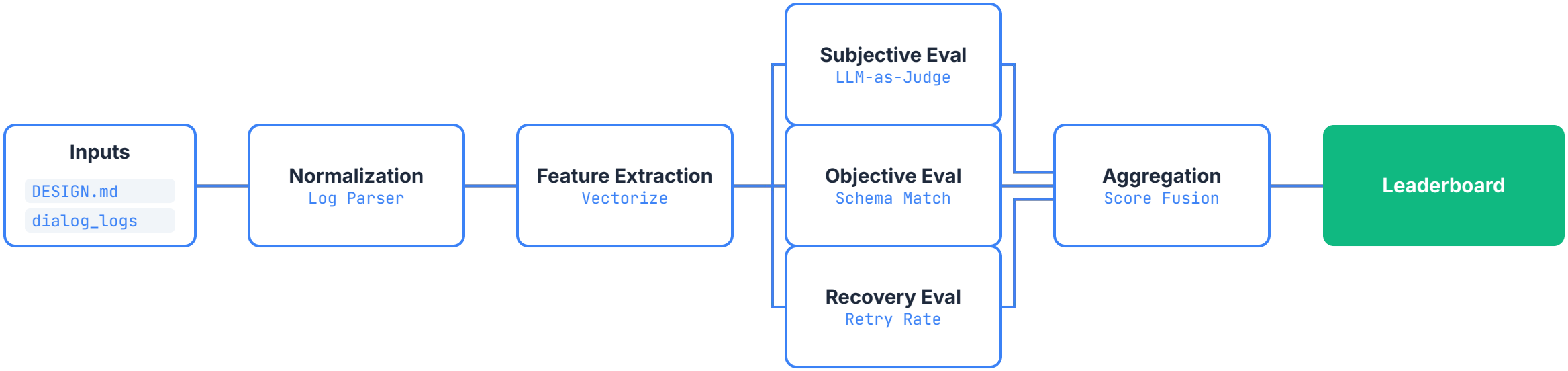
Runs independent MCP tool calls concurrently to reduce wall-clock latency in multi-agent diagnostic workflows.

3. Dynamic Specialist Routing and Replanning

Routes tasks to specialists and revises workflow dynamically based on tool results or failures, cutting wasted steps compared to fixed linear plan.

AssetOps Multi-turn Dialog Evaluation

The scoring pipeline automates the assessment of multi-turn interactions by validating rollout logs against ground-truth system designs. Through parallel subjective and objective evaluation branches, the system synthesizes raw execution data into a multi-dimensional performance radar.



Evaluation Metrics

Seven metrics covering **subjective quality** and **objective execution** to characterize model capabilities.

Notation: $D=\{d_1,..,d_n\}$; turns T_i; tool calls $C=\cup C_i$; Metric range $[0,1]$

Subjective Metrics

LLM-as-Judge / Human (Macro-Avg)

1. Planning Effectiveness

$$\text{PlanEff} = 1/N \Sigma\ s_plan_i$$

Probes: task decomposition, goal alignment, re-planning.

2. Tool Usage Quality

$$\text{ToolQual} = 1/N \Sigma\ s_tool_i$$

Probes: tool routing, argument construction, context use.

3. Task Completion

$$\text{TaskComp} = 1/N \Sigma\ s_task_i$$

Probes: end-to-end solving, correctness, deliverability.

Objective Metrics

Automated Log Parsing (Micro-Avg)

4. Tool Name Validity

$$\text{Validity} = \Sigma\ 1[\text{exists_c}] / |C|$$

Probes: tool-set boundary awareness, anti-hallucination.

5. Schema Compliance

$$\text{Schema} = \Sigma\ 1[\text{exists} \wedge \text{valid}] / \Sigma\ 1[\text{exists}]$$

Probes: structured generation, argument constraints.

6. Execution Success

$$\text{ExecSuccess} = \Sigma\ 1[\text{exec_ok_c}] / |C|$$

Probes: robustness of end-to-end tool orchestration.

Custom Metric

Dialog-Level Aggregation

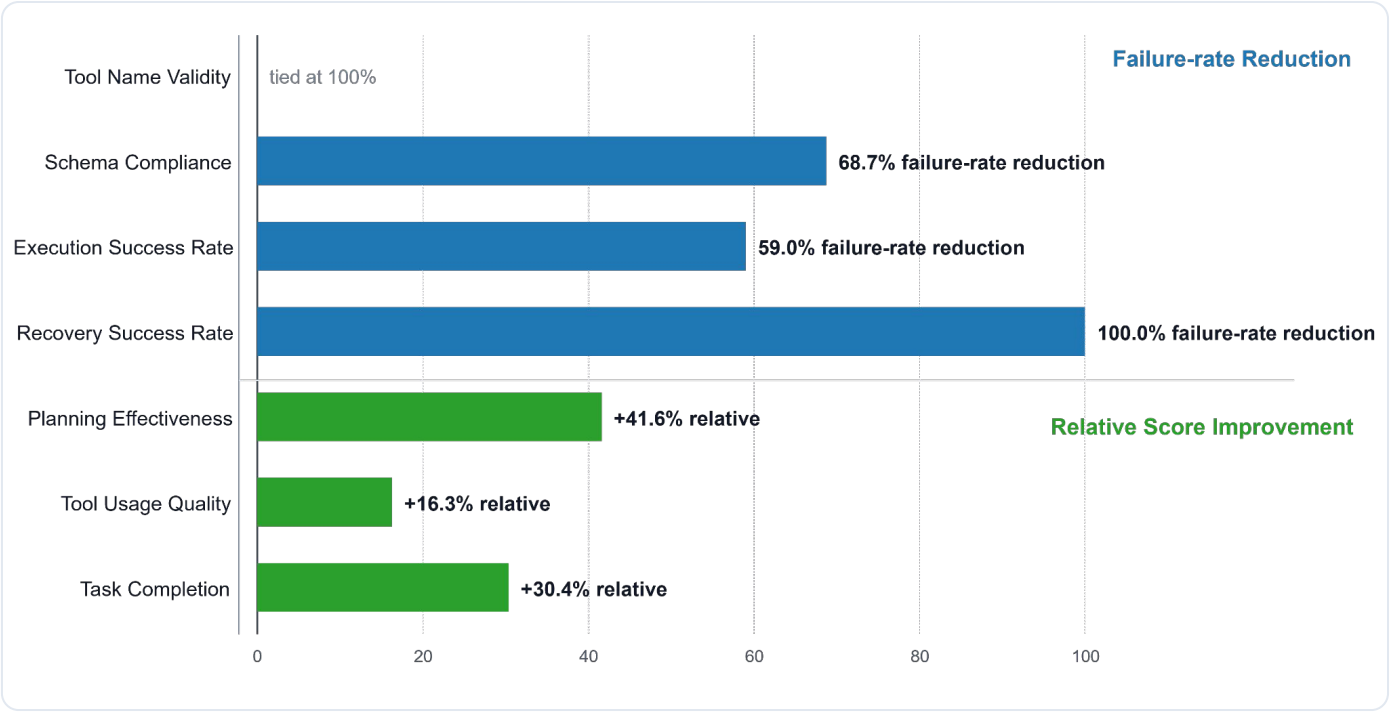
7. Recovery Success Rate

$$\text{RecoverySR} = |\{d_i \in D_rec : \text{success}(d_i)\}| / |D_rec|$$

Probes: error awareness and self-repair via retry.

Results — our system vs baseline





Per-metric improvement over baseline (blue = fewer failures, green = higher subjective score, grey = tied).

- **Call-level quality:** our model cuts schema failures by ~68.7% and execution failures by ~59.0% compared with baseline; recovery failures are fully eliminated.
- **Subjective uplift:** LLM-as-Judge scores rise across all axes — Planning +41.6%, Tool Usage +16.3%, Task Completion +30.4% relative.
- **Ceiling and saturation:** Tool Name Validity is tied at 100%, while Recovery Success Rate reaches 100%, leaving no remaining headroom on these axes.

Profiling Methodology

3-Layer Instrumentation Stack

Layer 1: LiteLLM Metrics
Prompt/Completion tokens and latency per LLM call.

Layer 2: MCP Tool Metrics
Record per tool invocation via _write_tool_metric() wrappers.

Layer 3: CouchDB Query Metrics
Latency and document return counts per db.find() call.

Tracking & Comparisons

Dashboards

- WandB: per-dialog and per-turn metrics, run-level summaries
- LangSmith: per-LLM-call traces; full graph traces on LangGraph

Measurements

- Wall-clock latency and 3-way breakdown (LLM / Tool / Routing)
- Token consumption: prompt, completion, per-turn efficiency
- MCP tool call counts, latency, and error rates — per server
- Turn success rate and dialog completion status

Architectures

- Baseline: Plan-Execute (single agent, sequential)
- SS: Supervisor-Specialist (LangGraph sequential)
- SSA: Supervisor-Specialist Advanced (Parallel MCP)

Results — Headline Numbers

1.9x

Faster End-to-End

Baseline 323.5s → SS 265.5s avg per dialog

4.3x

TSFM Tool Speedup

Baseline 159.5s → SS 37.4s avg per dialog

4.2×

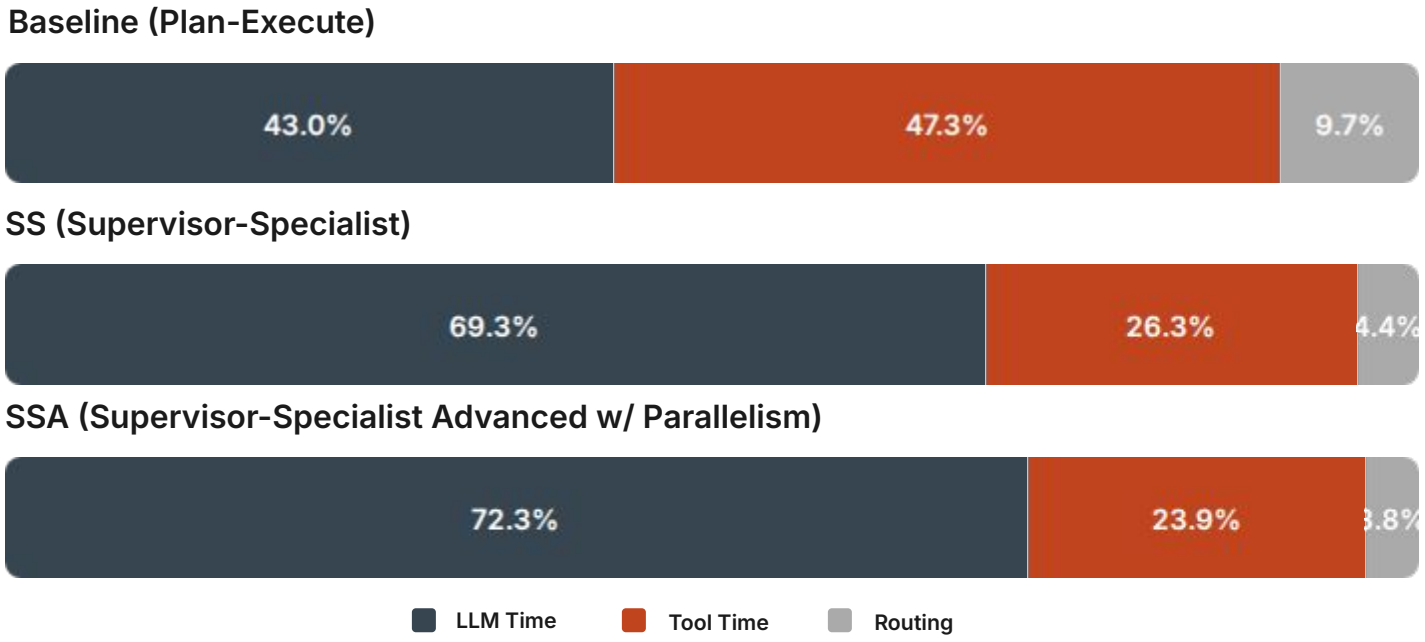
faster after turn 1

SS turns 2–5 avg 34s vs turn 1 at 145s. graph warms up fast

Measurements averaged over 16 benchmark dialogs on IBM Watsonx / Llama-4-Maverick-17B-FP8. No single architecture dominates on robustness as each fails on distinct scenarios. (Single run per architecture:completion counts reflect nondeterministic generation instead of structural reliability differences.)

Results By Architecture

Wall-Time Latency Breakdown (sec)



Per-Server Tool Latency (sec)

Server	Baseline	SS	SSA
TSFM	159.5s	**37.4s**	**36.1s**
IoT	24.9s	12.9s	11.8s
WO	12.4s	13.4s	12.9s
Utilities	4.0s	4.6s	4.1s
FMSR	9.9s	12.1s	19.7s

Takeaways

- Tool time drops **47% → 24%**; supervisor routing, not parallelism, is the driver.
- **LLM API latency** is the new ceiling (70%+ of wall time).
- LangGraph routing overhead is negligible (**< 5%**).

- TSFM is the dominant bottleneck for Plan-Execute.
- Supervisor routing eliminates redundant time-series queries.

Profiler Evidence

Wandb Dashboards

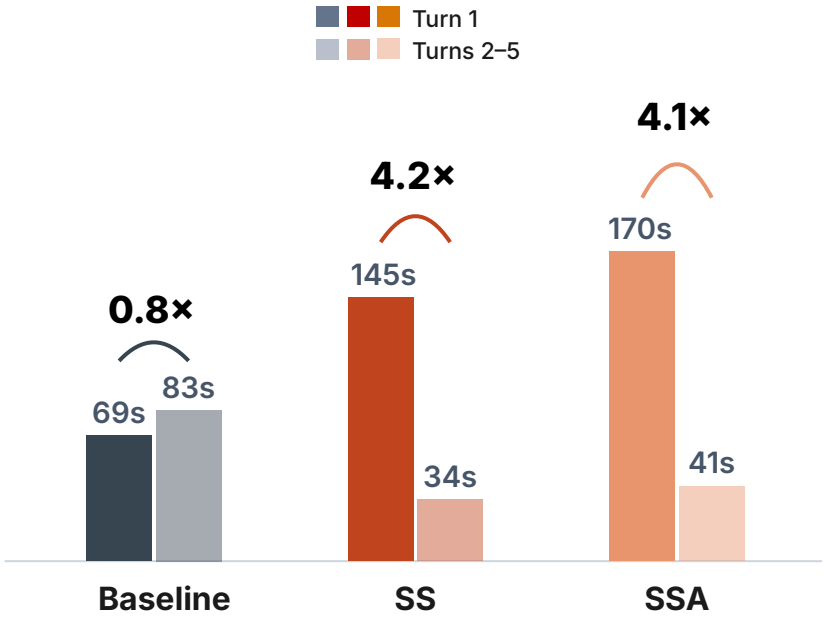
- **All Dialogs:** https://wandb.ai/rl3641-col/multi-turn-assetops?nw=lk02esfc7g
- **Single Dialog (D1):** https://wandb.ai/rl3641-col/multi-turn-assetops?nw=9r0g08ggnhi

Run-Level Summary

Metric	Baseline	SS	SSA
Total wall time	83.9 min	**65.2 min**	73.3 min
Total tokens consumed	**2,553,150**	3,322,234	3,623,430
Total LLM API calls	841	941	**751**

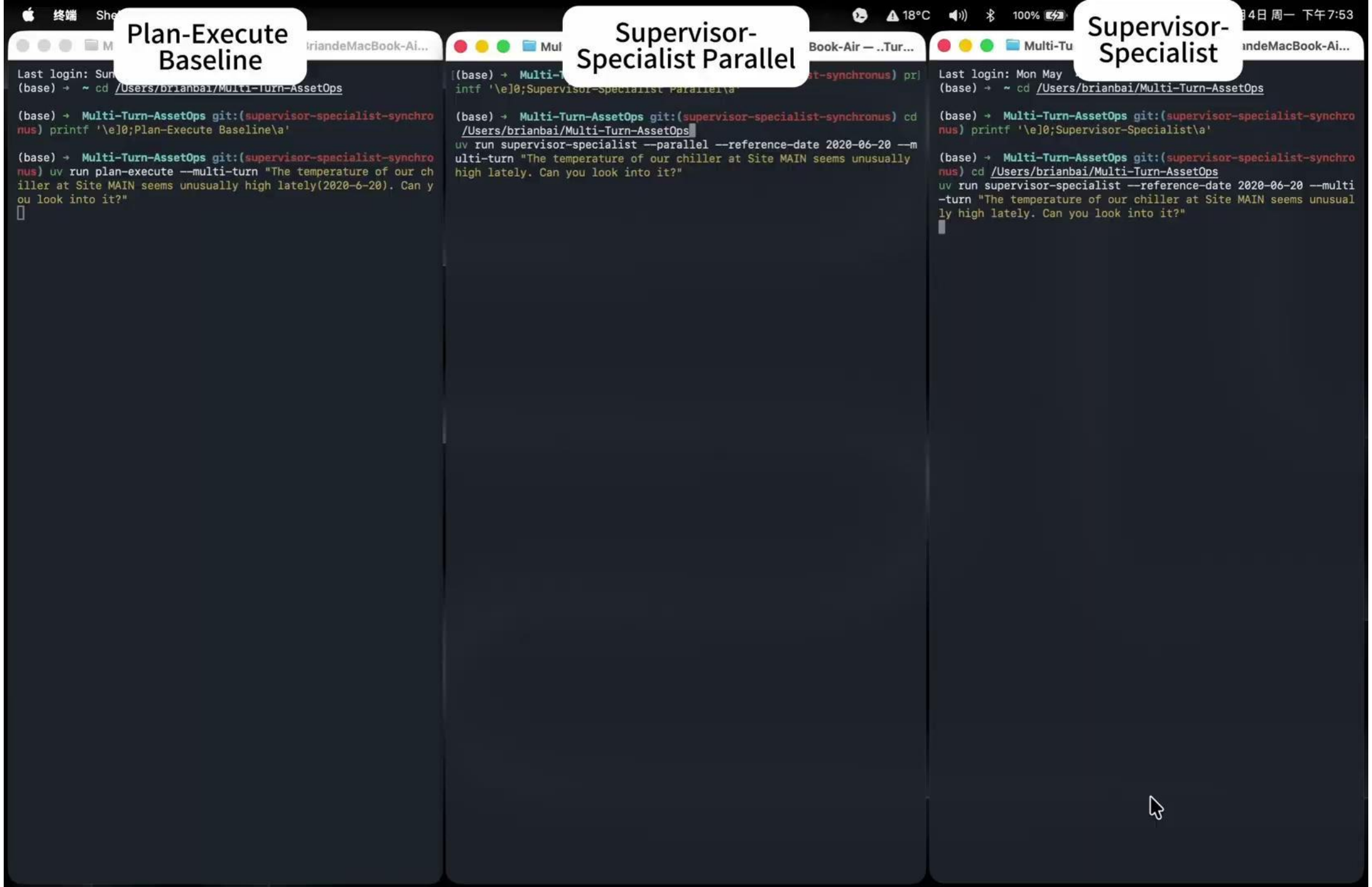
- SS is fastest end-to-end
- SSA token paradox

Turn 1 vs Turns 2–5 (mean duration)



- Supervisor cost is front-loaded into turn 1 (4.2×)
- Baseline accumulates cost across turns (turns get slower)

Demo / Visualization


Plan-Execute Baseline
Supervisor-Specialist Parallel
Supervisor-Specialist
Last login: Sun
(base) → ~ cd /Users/brianbai/Multi-Turn-AssetOps
(base) → Multi-Turn-AssetOps git:(supervisor-specialist-synchronus) printf '\e]0;Plan-Execute Baseline\a'
(base) → Multi-Turn-AssetOps git:(supervisor-specialist-synchronus) uv run plan-execute --multi-turn "The temperature of our chiller at Site MAIN seems unusually high lately(2020-6-20). Can you look into it?"
(base) → Multi-Turn-AssetOps git:(supervisor-specialist-synchronus) cd /Users/brianbai/Multi-Turn-AssetOps
uv run supervisor-specialist --parallel --reference-date 2020-06-20 --multi-turn "The temperature of our chiller at Site MAIN seems unusually high lately. Can you look into it?"
Last login: Mon May
(base) → ~ cd /Users/brianbai/Multi-Turn-AssetOps
(base) → Multi-Turn-AssetOps git:(supervisor-specialist-synchronus) printf '\e]0;Supervisor-Specialist\a'
(base) → Multi-Turn-AssetOps git:(supervisor-specialist-synchronus) cd /Users/brianbai/Multi-Turn-AssetOps
uv run supervisor-specialist --reference-date 2020-06-20 --multi-turn "The temperature of our chiller at Site MAIN seems unusually high lately. Can you look into it?"

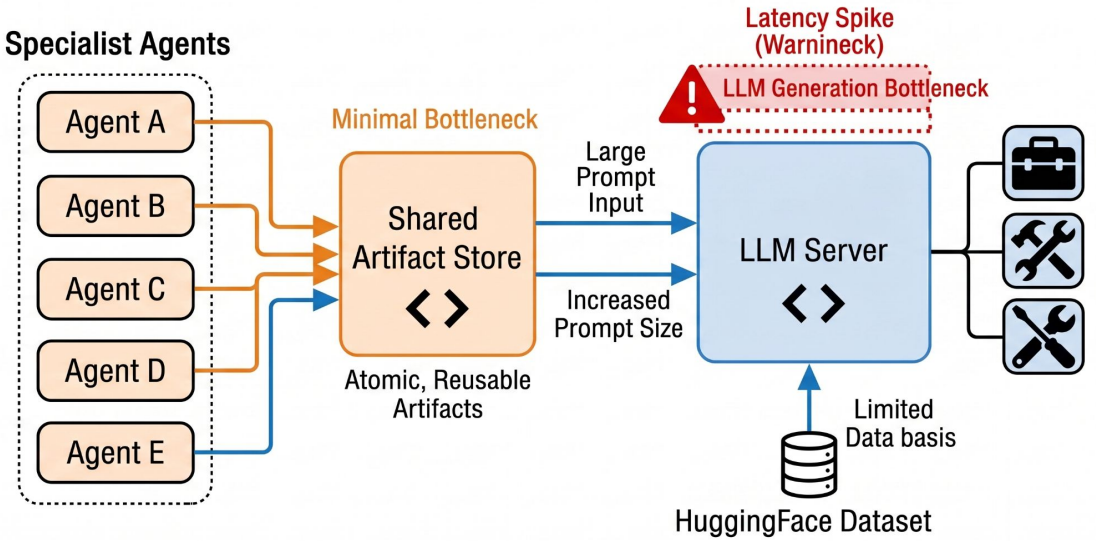
Lessons Learned

What worked

- Built a multi-agent system designed for complex multi-turn dialogue scenarios, significantly **improving response quality** over the baseline.
- **Identified tool invocation as the primary bottleneck**, and mitigated it through cross-turn artifact reuse.
- Decomposed the system into multiple specialists, preserving context purity and enabling the generation of **atomic, reusable artifacts**.
- Introduced no additional routing or orchestration overhead vs. plan-execute baseline, while **reducing repeated replanning** caused by low-quality initial plans.
- Structured logging **made failures visible**, especially missing assets, repeated tools, and bottleneck shifts.

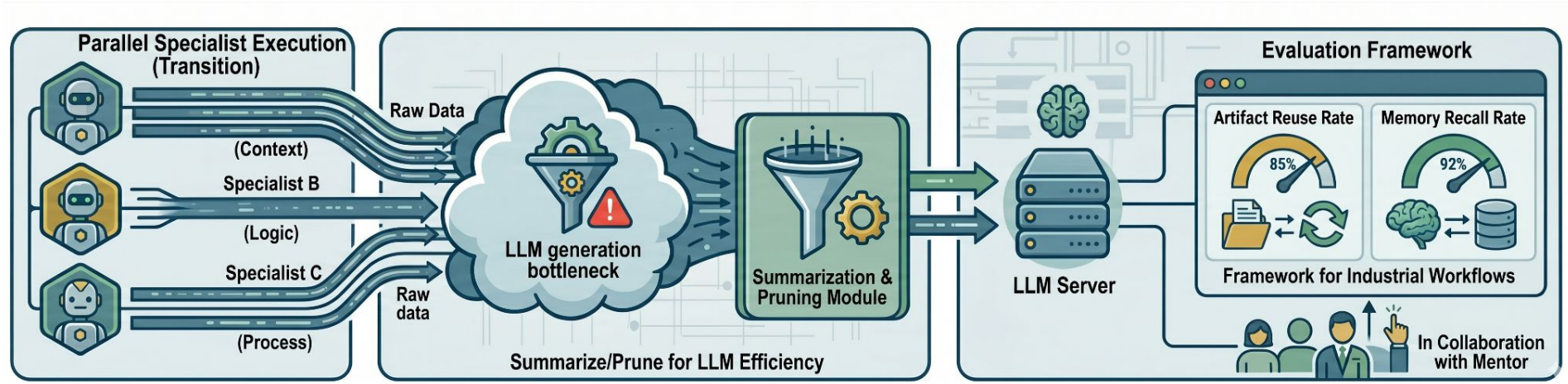
What didn't work

- Parallel tool execution **failed to reduce latency** as it **shifted the bottleneck to LLM generation** due to increased prompt size.
- Original MCP servers **lacked adaptable tooling for collaborative tasks**, limiting the upper bound of overall response quality.
- In the absence of **fully authentic industrial data and processes**, we can only use the HuggingFace dataset as a basis for further development.



Next Steps

- Transition from parallel tool invocation to **parallel specialist execution**.
- Further analyze the **LLM generation bottleneck** caused by parallel tool execution. Since parallel calls return more results at once, a potential solution is to summarize or prune the tool outputs.
- Build a multi-turn dialog system evaluation framework **for industrial workflows** based on key metrics like *artifact reuse and memory recall rate*, in collaboration with our mentor.



Teamwork & Contributions

Time budget: 30 seconds. Who did what. Required slide — graded individually.

Member	Primary contributions	Tools / artifacts
Rujing Li	Scenario design, profiling pipeline, results analysis	eval/*, wandb dashboards
Yitong Bai	Multi-agent architecture optimization techniques	src/*
Chengrui Li	Baseline refactoring, multi-agent architecture setup	src/*
Rui Li	Evaluation pipeline, results analysis, figures	evaluation_system/*

Collaboration: regular meetings via Zoom / in person every week

Thank you

Questions?

GitHub: https://github.com/Coderlicr/Multi-Turn-AssetOps
Tracking: https://wandb.ai/rl3641-col/multi-turn-assetops?nw=lk02esfc7g

Backup — Evaluation Leaderboard

Backup slide. Use for Q&A. Performance comparison of core architectural patterns.

Model	Planning Eff.	Tool Quality	Task Comp.	Name Validity	Schema Comp.	Exec Success	Recovery Rate
Plan-Execute	0.559	0.731	0.709	1.000	0.680	0.648	0.850
Supervisor-Specialist	**0.791**	**0.850**	**0.925**	**1.000**	**0.900**	**0.856**	**1.000**

Backup — Architecture Comparison

Backup slide. Use for Q&A. Baseline system.

1. Baseline: Single-Agent / Plan-Execute

